

Laboratorio 14 · Resolución de incidencias

Duración: 30 minutos · **Carpeta de trabajo:** entorno/

Objetivos

Vas a resolver cuatro incidencias típicas usando solo las herramientas incluidas en MongoDB: los logs estructurados, `mongostat`, `mongotop`, el profiler, `explain()` y `currentOp()`. La idea es seguir siempre el mismo método: **síntoma** → **evidencia** → **causa** → **solución** → **verificación**.

Preparación

```
cd entorno
docker compose up -d
docker compose exec toolbox bash /scripts/estado.sh
```

Necesitas dos terminales dentro del toolbox (`docker compose exec toolbox bash` en cada una).

Incidencia 1 · "La pantalla de pedidos del cliente va lentísima"

Evidencias

Terminal A: lanza la carga de trabajo.

```
mongosh "$RS" --quiet --eval "var MINUTOS=15" /scripts/carga.js
```

Terminal B: observa el servidor en tiempo real con `mongostat` y `mongotop` (Ctrl+C para salir de cada uno):

```
mongostat --uri "$RS" --discover 2
mongotop --uri "$RS" 5
```

Ahora busca las operaciones lentas. El profiler guarda las que superan el umbral; si no lo activaste en el laboratorio 13, actívalo ahora en el primario:

```
mongosh "$RS"
```

```

> use tienda
> db.setProfilingLevel(1, { slows: 50 })
> db.system.profile.find({ ns: "tienda.pedidos" }, { op: 1, command: 1,
millis: 1, docsExamined: 1, nreturned: 1, planSummary: 1 }).sort({ ts:
-1 }).limit(5)
> db.system.profile.aggregate([
  { $match: { ns: "tienda.pedidos" } },
  { $group: { _id: "$planSummary", veces: { $sum: 1 }, ms_medio: {
$avg: "$millis" }, examinados_medio: { $avg: "$docsExamined" } } },
  { $sort: { veces: -1 } }
])

```

Analiza una de las consultas con `explain`:

```

> db.pedidos.find({ cliente_id: 1234
}).explain("executionStats").executionStats
> db.pedidos.find({ estado: "enviado", ciudad: "Madrid" }).sort({ fecha:
-1 }).limit(20).explain("executionStats").queryPlanner.winningPlan

```

Tu diagnóstico y solución

Decide qué índices crear. Pista: para la segunda consulta aplica la regla **ESR** (*Equality, Sort, Range*).

▼ Ver solución

Ambas consultas hacen `COLLSCAN`: examinan los 200.000 documentos para devolver unos pocos. Solución:

```

> db.pedidos.createIndex({ cliente_id: 1 })
> db.pedidos.createIndex({ estado: 1, ciudad: 1, fecha: -1 })

```

En el segundo índice, primero van los campos de igualdad (estado , ciudad) y después el de ordenación (fecha), así el `sort` se resuelve recorriendo el índice en orden y no hace falta ordenar en memoria (desaparece la etapa `SORT`). Verificación: repite los `explain`; ahora verás `IXSCAN`, `totalDocsExamined` igual o muy cercano a `nReturned`, y en `mongostat` o en `PMM` bajará el tiempo de respuesta. En producción, crea los índices fuera de las horas punta: la construcción de índices se hace en todos los miembros del replica set a la vez y consume CPU y E/S.

Incidencia 2 · "Hay una operación que lleva minutos y está bloqueando todo"

Terminal A: para la carga (Ctrl+C) y lanza una operación deliberadamente lenta. La función `sleep` dentro de `$where` hace que cada documento tarde 100 ms en evaluarse (`$where` y el JavaScript en el servidor están desaconsejados; aquí los usamos solo porque son la forma más sencilla de fabricar una operación lenta, de unos 30 segundos):

```
mongosh "$RS" --quiet --eval 'db.getSiblingDB("tienda").pedidos.find({
$where: "sleep(100) || true" }).limit(300).toArray()'
```

Terminal B: encuéntrala y máatala.

```
> db.currentOp({ active: true, secs_running: { $gte: 3 }
}).inprog.map(op => ({ opid: op.opid, seg: op.secs_running, ns: op.ns,
op: op.op, cliente: op.client, filtro: op.command.filter })))
```

▼ Ver solución

```
> db.killOp(<opid>)
```

En la terminal A la consulta termina con el error `Interrupted (operation was interrupted)`. Antes de matar una operación en producción, identifica bien quién la ha lanzado (campo `cliente`, `appName`, `effectiveUsers`) y qué es: matar una construcción de índice o una operación interna del replica set puede tener consecuencias. Para evitar que se repita: `maxTimeMS` en la aplicación o `defaultMaxTimeMS` en el cluster (laboratorio 08).

Incidencia 3 · "Los informes que leen de los secundarios muestran datos viejos"

Simula un secundario que se ha quedado atrás bloqueándolo (en la práctica: disco lento, red saturada o un nodo sobrecargado). Desde tu equipo:

```
docker compose exec toolbox bash /scripts/primario.sh
docker compose exec toolbox bash /scripts/fsync.sh lock mongo3      #
usa un secundario
```

Terminal A: lanza de nuevo la carga para que haya escrituras:

```
mongosh "$RS" --quiet --eval "var MINUTOS=5" /scripts/carga.js
```

Terminal B: diagnostica.

```
> rs.printSecondaryReplicationInfo()  
> rs.status().members.map(m => ({ nodo: m.name, estado: m.stateStr,  
  optime: m.optimeDate, ultimoHeartbeat: m.lastHeartbeat })))
```

Busca también la evidencia en los logs del **primario** con `getLog`, que devuelve las últimas entradas del log estructurado:

```
> db.adminCommand({ getLog: "global" }).log.map(l =>  
  JSON.parse(l)).filter(e => e.s === "W" || e.s === "E").slice(-10).map(e  
    => ({ t: e.t.$date, c: e.c, id: e.id, msg: e.msg })))
```

Después desbloquea el nodo y comprueba cómo se recupera el lag:

```
docker compose exec toolbox bash /scripts/fsync.sh unlock mongo3  
docker compose exec toolbox bash /scripts/estado.sh
```

▼ Ver solución

`rs.printSecondaryReplicationInfo()` muestra que `mongo3` va cada vez más por detrás del primario, aunque su estado sigue siendo `SECONDARY` y los heartbeats son normales: el nodo está vivo pero no aplica el oplog. Esto es importante porque **un nodo con lag no dispara una alerta de "nodo caído"**: hace falta una alerta específica de lag. Con lag, las lecturas con `readPreference: secondary` devuelven datos obsoletos; si la aplicación lo tolera mal, hay que usar `maxStalenessSeconds` en la read preference para que el driver descarte los secundarios demasiado retrasados. Si el lag superase la ventana del oplog, el nodo pasaría a `RECOVERING` y necesitaría una resincronización completa.

Incidencia 4 · "La aplicación nueva no consigue conectarse"

Terminal A: simula una aplicación mal configurada.

```
for i in 1 2 3; do mongosh  
  "mongodb://app_tienda:ClaveEquivocada@mongo1,mongo2,mongo3/tienda?  
  replicaSet=rs0&authSource=admin&appName=facturacion" --quiet --eval  
  "db.pedidos.findOne()"; done
```

Terminal B: encuentra en los logs quién falla, desde dónde y por qué.

```
> db.adminCommand({ getLog: "global" }).log.map(l =>  
  JSON.parse(l)).filter(e => /Authentication
```

```
failed/i.test(e.msg)).slice(-3)
```

▼ Ver solución

Las entradas de `Authentication failed` incluyen el usuario (`user`), la base de datos de autenticación (`db`), el mecanismo, la dirección del cliente (`remote`) y el error (`AuthenticationFailed`). Como el log es JSON, se puede filtrar por campos concretos (`c` = componente, `id` = identificador del mensaje, `s` = severidad) en lugar de por texto, y enviar a cualquier herramienta de agregación de logs. Recuerda que el login se intenta contra el primario o contra cualquier miembro según la operación, así que en un replica set hay que revisar los logs de todos los nodos. Si el usuario `app_tienda` no existiera (por ejemplo, si no hiciste el laboratorio 08), el mensaje sería `UserNotFound`.

Chuleta de troubleshooting

Síntoma	Primera herramienta	Qué mirar
Lentitud general	<code>mongostat</code> , PMM	<code>qrw / arw</code> (colas), <code>dirty / used</code> de la caché, <code>conn</code>
Una consulta lenta	Profiler, <code>explain("executionStats")</code>	<code>COLLSCAN</code> , <code>docsExamined</code> frente a <code>nReturned</code> , etapa <code>SORT</code>
Colección "caliente"	<code>mongotop</code>	Tiempo de lectura/escritura por colección
Operación bloqueada	<code>db.currentOp()</code>	<code>secs_running</code> , <code>waitingForLock</code> , <code>client</code>
Lag de replicación	<code>rs.printSecondaryReplicationInfo()</code>	Diferencia de optime, heartbeats, carga del secundario
Errores de conexión o auth	Log (<code>getLog</code> o fichero)	<code>Authentication failed</code> , <code>Connection refused</code> , TLS
Arranque fallido	Log del proceso (<code>docker logs</code> , <code>journalctl</code>)	Mensajes de severidad <code>F</code> (fatal) y <code>E</code> (error)